

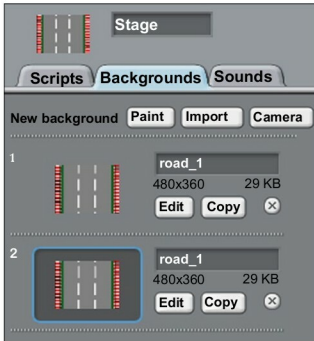


Figure 3: Code for the road background

respectively. The game gets over when the car meets with an accident or the fuel gets over. The points attained from the start of the game till the end are the final score.

Designing sprites using the Paint Editor

Open *Paint Editor*, which provides the user with basic drawing and designing tools like a brush, eraser, paint, etc. Now, let's design our first sprite, *car*.

The sprite, *car*, should have two costumes. The first is the default costume for playing, while the second costume, *crashed*, is used when the car collides with any of the obstacles. The user can also import images from the hard disk. Now, design the following sprites: *tank*, four obstacles, *steering_wheel* and *gear*. The sprite *steering_wheel* should have three costumes representing each orientation—left, right and straight. On clicking the left/right arrows, the sprite displays the costume corresponding to the direction. Similarly, the sprite, *gear*, consists of three costumes representing each shift.

Once created, the sprites will be displayed in the *Sprites List*. Then, click on the *Stage* icon and move on to the *Backgrounds* tab to design the background layer of the stage. Design two costumes of *road* with slight variations in the middle white lines.

Rearrange the sprites as shown in Figure 2.

Assigning codes for sprites and stage

In order to program the sprites, you need to drag the blocks (commands) into the scripts area of each sprite from the block palette.

Click on the *Make a variable* button under the *Variables* tab in the block palette to create the six variables – points, gear, gliding_time, fuel, time_gap and random. Click on *For all sprites* while declaring the variable so that they are declared as global variables.

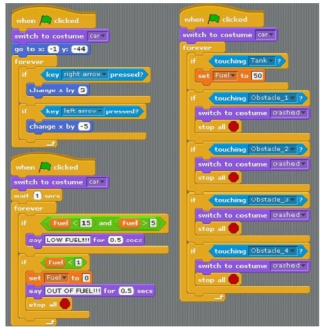


Figure 4: Code for the sprite car

Check the variables - *fuel*, *gear*, and *points*, so that they will be displayed on the stage.

The *fuel* variable indicates the amount of fuel the car has. It gets refilled as the car touches the *tank* sprite. The *gear* variable stores integer values from 1 to 3 (inclusive) representing each gear shift, and the speed of the *car* depends directly on the gear value. *Gliding_time* is used to change the speed of obstacles and the tank, according to the gear value. *Points* represent the score attained by the gamer. The *time_gap* variable is used as the time gap between each costume, in the road animation.

Click on the *stage* icon and drag down the scripts given in Figure 3 to the scripts area.

When the gamer clicks on the *Green flag* button, the game starts along with the initialisation of variables. Then the road animation starts with a time interval of the value stored in the *time_gap* variable. The value associated with the variables *time_gap* and *gliding_time* depends on the value of *gear*.

Now let's assign commands for the sprite *car*. Click on the sprite *car* and drag down the scripts given in Figure 4.

The *car* changes its x position as the gamer presses the right or left arrow. The game stops when the *car* touches any of the obstacles. The *Fuel* value gets incremented to 50 as the *car*

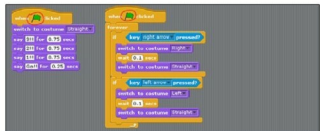


Figure 5: Code for the sprite steering wheel